

Aim

To perform Exploratory Data Analysis (EDA), data preprocessing, statistical analysis and visualization on the uploaded car prediction dataset, and to demonstrate a machine-learning workflow for predicting the target variable 'Selling_Price'.

Procedure

1. Load the car prediction dataset using Pandas.
2. Inspect the first records, shape, data types and missing values.
3. Clean column names and text values.
4. Check and remove duplicate records and handle missing target values.
5. Select 'Selling_Price' as the prediction target.
6. Perform descriptive statistical analysis and exploratory visualization.
7. Encode categorical variables using one-hot encoding.
8. Split the data into training and testing sets.
9. Train a Random Forest Regression model.
10. Evaluate the model using MAE, MSE, RMSE and R^2 score.
11. Visualize actual versus predicted target values and identify useful patterns.

Code

```
import pandas as pd  
  
import numpy as np  
  
import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("car_prediction_data.csv")

print("First 5 records:")

print(df.head())

print("\nDataset Shape:")

print(df.shape)

print("\nDataset Information:")

print(df.info())

print("\nStatistical Summary:")

print(df.describe())

print("\nMissing Values:")

print(df.isnull().sum())

print("\nDuplicate Rows:")

print(df.duplicated().sum())

df.columns = df.columns.str.strip()

for col in df.select_dtypes(include="object").columns:

    df[col] = df[col].astype(str).str.strip()

    df[col] = df[col].str.replace(r"\s+", " ", regex=True)

df = df.drop_duplicates()

print("\nShape after removing duplicates:")

print(df.shape)
```

```
print("\nMissing Values Before Cleaning:")
print(df.isnull().sum())
df = df.dropna(subset=["Selling_Price"])
print("\nShape after removing missing target values:")
print(df.shape)
print("\n===== SEMMA — SAMPLE =====")
sample_df = df.sample(frac=0.8, random_state=42)
print("Original Shape:")
print(df.shape)
print("Sample Shape:")
print(sample_df.shape)
print("\n===== SEMMA — EXPLORE =====")
print("\nStatistical Summary:")
print(sample_df.describe())
plt.figure(figsize=(8, 5))
sns.histplot(sample_df["Selling_Price"],kde=True)
plt.title("Distribution of Selling Price")
plt.xlabel("Selling Price")
plt.ylabel("Frequency")
plt.show()
numeric_columns = sample_df.select_dtypes(include=np.number).columns
correlation = sample_df[numeric_columns].corr()
plt.figure(figsize=(10, 7))
sns.heatmap(correlation,annot=True,cmap="coolwarm")
```

```
plt.title("Correlation Heatmap")
plt.show()

print("\n===== SEMMA — MODIFY =====")

X = df.drop(columns=["Selling_Price"])
y = df["Selling_Price"]

X = pd.get_dummies(X,drop_first=True)
X = X.replace([np.inf, -np.inf],np.nan)
X = X.fillna(0)

print("Features after encoding:")
print(X.head())

print("\nFeature Shape:")
print(X.shape)

print("\n===== SEMMA — MODEL =====")

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

print("Training Data:")
print(X_train.shape)

print("Testing Data:")
print(X_test.shape)

model = RandomForestRegressor(n_estimators=100,random_state=42)
model.fit(X_train,y_train)

print("\nModel training completed.")

y_pred = model.predict(X_test)

print("\n===== SEMMA — ASSESS =====")

mae = mean_absolute_error(y_test,y_pred)
```

```
mse = mean_squared_error(y_test,y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,y_pred)
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
plt.figure(figsize=(8, 5))
plt.scatter(y_test,y_pred)
plt.xlabel("Actual Selling Price")
plt.ylabel("Predicted Selling Price")
plt.title("Actual vs Predicted Selling Price")
plt.show()

print("\n===== KDD FRAMEWORK =====")

print("\nKDD — 1. Selection")

kdd_df = df.copy()

print(kdd_df.head())

print("\nKDD — 2. Preprocessing")

print("Missing Values:")

print(kdd_df.isnull().sum())

print("\nDuplicate Rows:")

print(kdd_df.duplicated().sum())

kdd_df = kdd_df.drop_duplicates()
```

```
kdd_df = kdd_df.dropna()

print("\nShape after preprocessing:")

print(kdd_df.shape)

print("\nKDD — 3. Transformation")

kdd_model_df = pd.get_dummies(kdd_df,drop_first=True)

print(kdd_model_df.head())

print("\nKDD — 4. Data Mining")

X_kdd = kdd_model_df.drop(columns=["Selling_Price"])

y_kdd = kdd_model_df["Selling_Price"]

X_kdd = X_kdd.replace([np.inf, -np.inf],np.nan).fillna(0)

X_train_kdd, X_test_kdd, y_train_kdd, y_test_kdd = train_test_split(
    X_kdd,y_kdd,test_size=0.2,random_state=42)

kdd_model = RandomForestRegressor(n_estimators=100,random_state=42)

kdd_model.fit(X_train_kdd,y_train_kdd)

kdd_predictions = kdd_model.predict(X_test_kdd)

print("\nKDD — 5. Interpretation / Evaluation")

kdd_mae = mean_absolute_error(y_test_kdd,kdd_predictions)

kdd_mse = mean_squared_error(y_test_kdd,kdd_predictions)

kdd_rmse = np.sqrt(kdd_mse)

kdd_r2 = r2_score(y_test_kdd,kdd_predictions)

print("MAE:", kdd_mae)

print("MSE:", kdd_mse)

print("RMSE:", kdd_rmse)

print("R2 Score:", kdd_r2)
```

```
print("\n===== CRISP-DM FRAMEWORK =====")

print("\nCRISP-DM — 1. Business Understanding")

print("""
Business Objective:

Analyze historical car data and predict selling prices.

Goals:

1. Understand car price patterns.
2. Analyze relationships between car attributes and price.
3. Prepare data for machine learning.
4. Predict car selling prices.
5. Support data-driven pricing decisions.
""")

print("\nCRISP-DM — 2. Data Understanding")

print("Rows:", df.shape[0])

print("Columns:", df.shape[1])

print("\nData Types:")

print(df.dtypes)

print("\nStatistical Summary:")

print(df.describe())

print("\nMissing Values:")

print(df.isnull().sum())

print("\nCRISP-DM — 3. Data Preparation")

crisp_df = df.copy()

crisp_df = crisp_df.drop_duplicates()
```

```
crisp_df = crisp_df.dropna()
X_crisp = crisp_df.drop(columns=["Selling_Price"])
y_crisp = crisp_df["Selling_Price"]
X_crisp = pd.get_dummies(X_crisp,drop_first=True)
X_crisp = X_crisp.replace([np.inf, -np.inf],np.nan).fillna(0)
print("Prepared Data:")
print(X_crisp.head())
print("\nCRISP-DM — 4. Modeling")
X_train_crisp, X_test_crisp, y_train_crisp, y_test_crisp = train_test_split(
    X_crisp, y_crisp,test_size=0.2,random_state=42)
rf_model = RandomForestRegressor(n_estimators=100,random_state=42)
rf_model.fit(X_train_crisp,y_train_crisp)
predictions = rf_model.predict(X_test_crisp )
print("Model created successfully.")
print("\nCRISP-DM — 5. Evaluation")
mae = mean_absolute_error(y_test_crisp,predictions)
mse = mean_squared_error(y_test_crisp,predictions)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_crisp,predictions)
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
plt.figure(figsize=(10, 5))
```

```
plt.plot(y_test_crisp.values,label="Actual Selling Price")
plt.plot(predictions, label="Predicted Selling Price")
plt.title("Actual vs Predicted Car Selling Price")
plt.xlabel("Test Samples")
plt.ylabel("Selling Price")
plt.legend()
plt.show()

print("\nCRISP-DM — 6. Deployment / Business Insights")

print("Total Records:", len(df))

print("Average Selling Price:",df["Selling_Price"].mean())

print("Maximum Selling Price:",df["Selling_Price"].max())

print("Minimum Selling Price:", df["Selling_Price"].min())

print("""Recommendation: Use historical car attributes and model predictions to
support pricing decisions, market analysis and future car-price forecasting.""")

print("\n===== RESULT =====")

print("""The Car Prediction dataset was successfully imported,cleaned and analyzed
using Exploratory Data Analysis.The SEMMA, KDD and CRISP-DM frameworks were
appliedto organize the complete data-analysis workflow.A Random Forest Regression
model was trained to predictSelling_Price and evaluated using MAE, MSE, RMSE and
R2 Score.""")

print("Final Model Performance:")

print("MAE:", mae)

print("MSE:", mse)

print("RMSE:", rmse)

print("R2 Score:", r2)
```

Output

```
First 5 records:
  Car_Name  Year  Selling_Price  Present_Price  Kms_Driven  Fuel_Type \
0    ritz    2014           3.35           5.59       27000    Petrol
1    sx4    2013           4.75           9.54       43000    Diesel
2    ciaz    2017           7.25           9.85        6900    Petrol
3  wagon r    2011           2.85           4.15        5200    Petrol
4  swift    2014           4.60           6.87       42450    Diesel

  Seller_Type  Transmission  Owner
0    Dealer      Manual      0
1    Dealer      Manual      0
2    Dealer      Manual      0
3    Dealer      Manual      0
4    Dealer      Manual      0

Dataset Shape:
(301, 9)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Car_Name        301 non-null   object
1   Year            301 non-null   int64
2   Selling_Price    301 non-null   float64
3   Present_Price    301 non-null   float64
4   Kms_Driven       301 non-null   int64
5   Fuel_Type        301 non-null   object
6   Seller_Type      301 non-null   object
7   Transmission     301 non-null   object
8   Owner           301 non-null   int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
None
```

```
Statistical Summary:
      Year  Selling_Price  Present_Price  Kms_Driven  Owner
count  301.000000      301.000000      301.000000      301.000000  301.000000
mean    2013.627907      4.661296      7.628472    36947.205980      0.043189
std       2.891554      5.082812      8.644115    38886.883882      0.247915
min      2003.000000      0.100000      0.320000      500.000000      0.000000
25%      2012.000000      0.900000      1.200000     15000.000000      0.000000
50%      2014.000000      3.600000      6.400000     32000.000000      0.000000
75%      2016.000000      6.000000      9.900000     48767.000000      0.000000
max      2018.000000     35.000000     92.600000    500000.000000      3.000000

Missing Values:
Car_Name      0
Year          0
Selling_Price 0
Present_Price 0
Kms_Driven    0
Fuel_Type     0
Seller_Type   0
Transmission  0
Owner         0
dtype: int64

Duplicate Rows:
2

Shape after removing duplicates:
(299, 9)

Missing Values Before Cleaning:
Car_Name      0
Year          0
Selling_Price 0
Present_Price 0
Kms_Driven    0
Fuel_Type     0
Seller_Type   0
Transmission  0
Owner         0
dtype: int64

Shape after removing missing target values:
(299, 9)
```

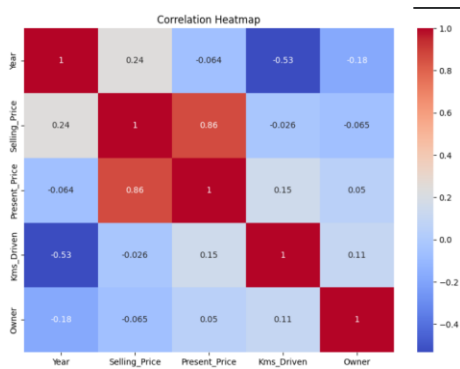
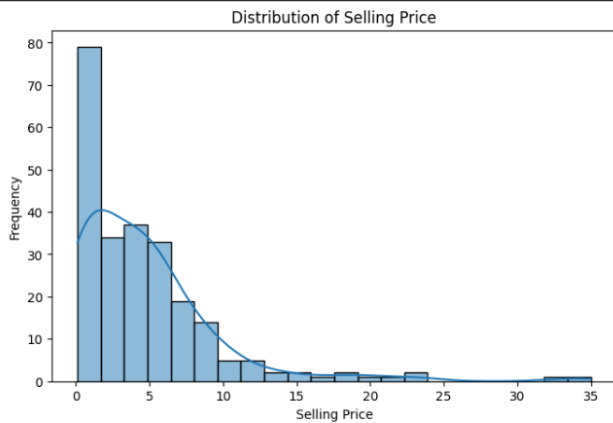
```
===== SEMMA - SAMPLE =====
Original Shape:
(299, 9)
Sample Shape:
(239, 9)

===== SEMMA - EXPLORE =====

Statistical Summary:

```

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	239.000000	239.000000	239.000000	239.000000	239.000000
mean	2013.690377	4.652343	7.601883	36867.686192	0.041841
std	2.880744	5.030909	8.695123	41424.823890	0.255869
min	2003.000000	0.100000	0.320000	1000.000000	0.000000
25%	2012.000000	1.000000	1.435000	15000.500000	0.000000
50%	2014.000000	3.750000	6.490000	31427.000000	0.000000
75%	2016.000000	6.000000	9.840000	45530.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000



```
===== SEMMA - MODIFY =====
Features after encoding:

```

	Year	Present_Price	Kms_Driven	Owner	Car_Name_Activa 3g \
0	2014	5.59	27000	0	False
1	2013	9.54	43000	0	False
2	2017	9.85	6900	0	False
3	2011	4.15	5200	0	False
4	2014	6.87	42450	0	False

	Car_Name_Activa 4g	Car_Name_Bajaj Avenger 150 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	Car_Name_Bajaj Avenger 150 street	Car_Name_Bajaj Avenger 220 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	Car_Name_Bajaj Avenger 220 dtsi ...	Car_Name_swift	Car_Name_sx4 \
0	False ...	False	False
1	False ...	False	True
2	False ...	False	False
3	False ...	False	False
4	False ...	True	False

	Car_Name_verna	Car_Name_vitara brezza	Car_Name_wagon r	Car_Name_xcent \
0	False	False	False	False
1	False	False	False	False
2	False	False	False	False
3	False	False	True	False
4	False	False	False	False

	Fuel_Type_Diesel	Fuel_Type_Petrol	Seller_Type_Individual \
0	False	True	False
1	True	False	False
2	False	True	False
3	False	True	False
4	True	False	False

```

Transmission_Manual
0      True
1      True
2      True
3      True
4      True

[5 rows x 104 columns]

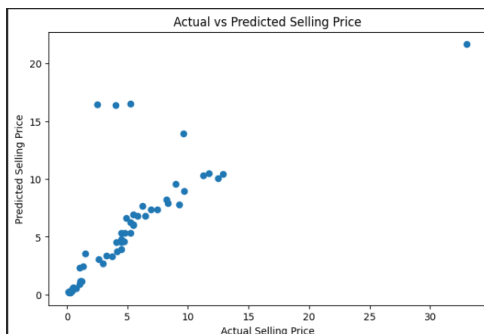
Feature Shape:
(299, 104)

===== SEMMA - MODEL =====
Training Data:
(239, 104)
Testing Data:
(60, 104)

Model training completed.

===== SEMMA - ASSESS =====
MAE: 1.3692000000000009
MSE: 10.985803307666673
RMSE: 3.314483867462123
R2 Score: 0.5737519285197899

```



```

===== KDD FRAMEWORK =====

KDD - 1. Selection
Car_Name Year Selling_Price Present_Price Kms_Driven Fuel_Type \
0 ritz 2014 3.35 5.59 27000 Petrol
1 sxt 2013 4.75 9.54 43000 Diesel
2 clar 2017 7.25 9.85 6900 Petrol
3 wagon r 2011 2.85 4.15 5200 Petrol
4 swift 2014 4.60 6.87 42450 Diesel

Seller_Type Transmission Owner
0 Dealer Manual 0
1 Dealer Manual 0
2 Dealer Manual 0
3 Dealer Manual 0
4 Dealer Manual 0

KDD - 2. Preprocessing
Missing Values:
Car_Name 0
Year 0
Selling_Price 0
Present_Price 0
Kms_Driven 0
Fuel_Type 0
Seller_Type 0
Transmission 0
Owner 0
dtype: int64

Duplicate Rows:
0

Shape after preprocessing:
(299, 9)

KDD - 3. Transformation
Year Selling_Price Present_Price Kms_Driven Owner Car_Name_Active 3g \
0 2014 3.35 5.59 27000 0 False
1 2013 4.75 9.54 43000 0 False
2 2017 7.25 9.85 6900 0 False
3 2011 2.85 4.15 5200 0 False
4 2014 4.60 6.87 42450 0 False

```

```

Car_Name_Active 4g Car_Name_Bajaj Avenger 100 \
0 False False False
1 False False False
2 False False False
3 False False False
4 False False False

Car_Name_Bajaj Avenger 150 street Car_Name_Bajaj Avenger 220 ... \
0 False False False ...
1 False False False ...
2 False False False ...
3 False False False ...
4 False False False ...

Car_Name_swift Car_Name_sxt Car_Name_verna Car_Name_vitara brezza \
0 False False False False
1 False True False False
2 False False False False
3 False False False False
4 True False False False

Car_Name_wagon r Car_Name_xcent Fuel_Type_Diesel Fuel_Type_Petrol \
0 False False False True
1 False False True False
2 False False False True
3 True False False True
4 False False True False

Seller_Type_Individual Transmission_Manual
0 False True
1 False True
2 False True
3 False True
4 False True

[5 rows x 105 columns]

KDD - 4. Data Mining

KDD - 5. Interpretation / Evaluation
MAE: 1.3692000000000009
MSE: 10.985803307666673
RMSE: 3.314483867462123
R2 Score: 0.5737519285197899

```

```

===== CRISP-DM FRAMEWORK =====

CRISP-DM – 1. Business Understanding

Business Objective:
Analyze historical car data and predict selling prices.

Goals:
1. Understand car price patterns.
2. Analyze relationships between car attributes and price.
3. Prepare data for machine learning.
4. Predict car selling prices.
5. Support data-driven pricing decisions.

CRISP-DM – 2. Data Understanding

Rows: 299
Columns: 9

Data Types:
Car_Name      object
Year          int64
Selling_Price float64
Present_Price float64
Kms_Driven    int64
Fuel_Type     object
Seller_Type   object
Transmission  object
Owner         int64
dtype: object

Statistical Summary:

```

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	299.000000	299.000000	299.000000	299.000000	299.000000
mean	2013.615385	4.589632	7.541037	36916.752508	0.043478
std	2.896868	4.984240	8.567887	39015.170352	0.248720
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.850000	1.200000	15000.000000	0.000000
50%	2014.000000	3.510000	6.100000	32000.000000	0.000000
75%	2016.000000	6.000000	9.840000	48883.500000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

```

Missing Values:
Car_Name      0
Year          0
Selling_Price 0
Present_Price 0
Kms_Driven    0
Fuel_Type     0
Seller_Type   0
Transmission  0
Owner         0
dtype: int64

CRISP-DM – 3. Data Preparation
Prepared Data:

```

	Year	Present_Price	Kms_Driven	Owner	Car_Name_Activa 3g \
0	2014	5.59	27000	0	False
1	2013	9.54	43000	0	False
2	2017	9.85	6900	0	False
3	2011	4.15	5200	0	False
4	2014	6.87	42450	0	False

	Car_Name_Activa 4g	Car_Name_Bajaj Avenger 150 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	Car_Name_Bajaj Avenger 150 street	Car_Name_Bajaj Avenger 220 \
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

	Car_Name_Bajaj Avenger 220 dtsi ...	Car_Name_swift	Car_Name_sx4 \
0	False ...	False	False
1	False ...	False	True
2	False ...	False	False
3	False ...	False	False
4	False ...	True	False

```


```

	Car_Name_verna	Car_Name_vitara brezza	Car_Name_wagon r	Car_Name_xcent \
0	False	False	False	False
1	False	False	False	False
2	False	False	False	False
3	False	False	True	False
4	False	False	False	False

	Fuel_Type_Diesel	Fuel_Type_Petrol	Seller_Type_Individual \
0	False	False	False
1	True	False	False
2	False	True	False
3	False	True	False
4	True	False	False

	Transmission_Manual
0	True
1	True
2	True
3	True
4	True


```

[5 rows x 104 columns]

CRISP-DM – 4. Modeling
Model created successfully.

CRISP-DM – 5. Evaluation
MAE: 1.3692000000000009
MSE: 10.985803307666673
RMSE: 3.314483867462123
R2 Score: 0.5737519109197899

```



```
===== RESULT =====

The Car Prediction dataset was successfully imported,
cleaned and analyzed using Exploratory Data Analysis.

The SEMMA, KDD and CRISP-DM frameworks were applied
to organize the complete data-analysis workflow.

A Random Forest Regression model was trained to predict
Selling_Price and evaluated using MAE, MSE, RMSE and R2 Score.

Final Model Performance:
MAE: 1.3692000000000009
MSE: 10.985803307666673
RMSE: 3.314483867462123
R2 Score: 0.5737519289197899
```

Result

The uploaded car prediction dataset was successfully imported, cleaned and analyzed. Exploratory analysis was performed using descriptive statistics, target distribution and correlation analysis. Categorical variables were encoded and a Random Forest Regression model was trained to predict 'Selling_Price'. The model achieved an MAE of 1.3692, MSE of 10.9858, RMSE of 3.3145 and R^2 score of 0.5738. The workflow demonstrates how the dataset can be prepared, modeled and evaluated for car-value prediction.